

SECP3623 Database Programming

Assignment 2 - Transaction Management

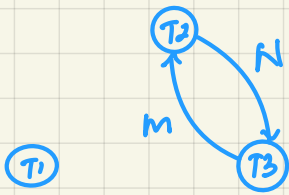
Name : Chan Ying Jia (A23CS0213)

Cheryl Cheong Kah Voon (A23CS0060)

Section : 01

Question 1

i)

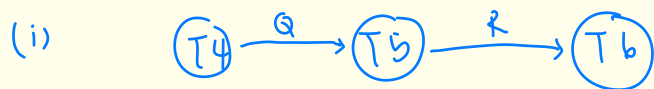


ii) The precedence graph contains a cycle between T2 and T3
 If a cycle exist, the schedule is not serializable. It is non-conflict serializable.

Time	T ₁	T ₂	T ₃
t ₁	begin transaction		
t ₂	S-lock(X)	begin transaction	
t ₃	read_item(X)	X-lock(M)	begin transaction
t ₄	S-lock(Y)	read_item(M)	X-lock(N)
t ₅	read_item(Y)	Request S-lock(N)	read_item(N)
t ₆	commit, unlock(X,Y)		Request S-lock(M)
t ₇			
t ₈		DEADLOCK	DEADLOCK.

iv) There exist a deadlock in both T₂ and T₃.
 Transaction T₂ holds X-lock(M) and is waiting for a lock on N.
 Meanwhile, T₃ holds X-lock(N) and is waiting for a lock on M.
 Neither transaction can proceed as they are stuck in a circular wait.

Question 2



(ii) The precedence graph is constructed whether the given transaction schedule is conflict-serializable. In the graph, each node represents a transaction, T_4 , T_5 and T_6 , and a directed edge $T_i \rightarrow T_j(x)$ is added if T_i read/write the data before T_j due to a conflicting operation in some way as same execution.

(iii)

Table 2: Transaction Schedule for T_4 - T_5 - T_6

	T4	T5	T6
↓ time	begin transaction		
	read_item(P)		
	read_item(Q)	begin transaction	
	$P = (P * 5) + Q$	read_item(Q)	
	write_item(P)	read_item(R)	begin transaction
	commit	$Q = Q + R$	read_item(R)
		write_item(Q)	$R = R - 100$
		commit	write_item(R)
			commit

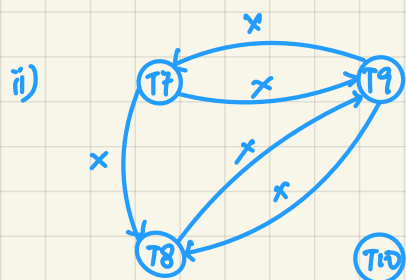
(ii)	Time	T4	T5	T6
	t_1	begin transaction		
	t_2	write_lock (P)		
	t_3	read_item (P)	begin transaction	
	t_4	write_lock (Q)	write_lock (Q)	
	t_5	read_item (Q)	wait	begin transaction
	t_6	$P = (P * 5) + Q$	wait	write_lock (R)
	t_7	write_item (P)	wait	read_item (R)
	t_8	commit / unlock (P, Q)	wait	$R = R - 100$
	t_9		read_item (Q)	write_item (R)
	t_{10}		write_lock (R)	commit / unlock (R)
	t_{11}		read_item (R)	
	t_{12}		$Q = Q + R$	
	t_{13}		write_item (Q)	
	t_{14}		commit / unlock (Q, R)	

Question 8.

a. i) It is a non-serial schedule.

The operations of the schedules are interleaved.

If it is serial, transactions will be executed one after another without interleaving.



iii) The schedule is non-conflict serializable.

There is a cycle between T7, T8 and T9 based on the precedence graph.

It cannot be serialized.

iv) Schedule in table 3.1 faces the lost update problem.

T9 reads x at t_3 but before it writes its update in t_6 , T7 has written a new value in t_5 .

T9 then overwrites T7's update using the old value of x at t_6 .

This overwriting causes the lost update problem.

b. i) Time	T1	T2	T3	T4
t_1	begin transaction.			
t_2	$x\text{-lock}(A)$	begin transaction.		
t_3	$\text{read}(A)$	$x\text{-lock}(B)$	begin transaction.	
t_4		$\text{read}(B)$	request $x\text{-lock}(A)$	begin transaction.
t_5	$A = A + 200$	$B = B - 50$		$x\text{-lock}(C)$
t_6	$\text{write}(A)$			$\text{read}(C)$
t_7	$\text{unlock}(A)$	$\text{write}(B)$		$C = C + 10$
t_8		$\text{unlock}(B)$	$x\text{-lock}(A)$	$\text{write}(C)$
t_9			$\text{read}(A)$	$\text{unlock}(C)$
t_{10}	commit			
t_{11}		request $x\text{-lock}(A)$	$A = A - 30$	
t_{12}			$\text{write}(A)$	
t_{13}			$\text{unlock}(A)$	
t_{14}		$x\text{-lock}(A)$		commit.
t_{15}		$\text{read}(A)$		
t_{16}		$A = A \times 2$		
t_{17}		$\text{write}(A)$		
t_{18}		commit / $\text{unlock}(A)$	commit.	

2b. ii) The issue arising is starvation. T2 has to wait until all $x\text{-lock}(A)$ is released at t_{10} .

iii) Time	T11	T12	T13	T14.
t_1	begin transaction			
t_2	write_lock(A)	begin transaction		
t_3	read(A)	write_lock(B)	begin transaction	
t_4		read(B)	write_lock(A)	begin transaction
t_5	$A = A + 200$	$B = B - 50$	WAIT	write_lock(C)
t_6	write(A)		WAIT	read(C)
t_7		write(B)	WAIT	$C = C + 10$
t_8			WAIT	write(C)
t_9	commit / unlock(A)		WAIT	
t_{10}		write_lock(A)	read(A)	
t_{11}		WAIT		
t_{12}		WAIT	$A = A - 30$	
t_{13}		WAIT	write(A)	commit / unlock(C)
t_{14}		WAIT		
t_{15}		WAIT		
t_{16}		WAIT		
t_{17}		WAIT		
t_{18}		WAIT	commit / unlock(A)	
t_{19}		read(A)		
t_{20}		$A = A * 2$		
t_{21}		write(A)		
t_{22}		commit / unlock(A)		

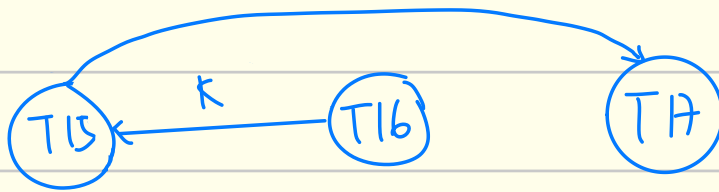
iv) There is still a starvation problem that looks more trouble.

It is now more likely a serial transaction that waits one of the transaction to finish first. The waiting transaction, T12, have to hold start for 8 time stamps

Question 4

(i) the schedule in Table 4 is non-serial schedule. A serial-schedule is where operations of each transaction are executed consecutively without any interleaved operation from other transactions. In table 4, transaction, T15, T16, T17 are interleaved because T16 and T17 begin and execute before T15 commits.

(ii)



(iii) Yes, it has conflict-serializable. The precedence graph has no cycle. So it is conflict-serializable.

(iv) lost update. When T15 are update and write J, at the same time, T17 read J with the old value and J is overwrite with the old value.

Table 4.1: Schedule T15, T16 and T17

Time	T15	T16	T17
t_1	begin transaction		
t_2	read (J)	begin transaction	
t_3	$J = J + 10$	read (K)	begin transaction
t_4	write (J)	$K = K - 5$	read (J)
t_5	Read (K)	write (K)	$J = J - 15$
t_6	$K = K + 20$		write (J)
t_7	write (K)	commit	
t_8	commit		commit

(V) Time	T1S	T1b	T1f
t ₁	begin transaction		
t ₂	write_lock(J)	begin transaction	
t ₃	read(J)	write_lock(K)	begin transaction
t ₄	J = J + 10	read(K)	write_lock(J)
t ₅	write(J)	K = K - 5	wait
t ₆	write_lock(K)	write(K)	wait
t ₇	wait	commit/unlock(K)	wait
t ₈	read(K)		wait
t ₉	K = K + 20		wait
t ₁₀	write(K)		wait
t ₁₁	commit/unlock(J, K)		wait
t ₁₂			read(J)
t ₁₃			J = J - 15
t ₁₄			write(J)
t ₁₅			commit/unlock(J)